



Pertemuan 7

▼ Multi Layout & Nested Routes

Pertemuan kali ini kita akan coba menggabungkan 2 komponen layout yang berbeda:

- Komponen `AuthLayout` → menampilkan halaman login, forgot password, reset password dll
- Komponen `MainLayout` → layout utama yang menampilkan sidebar, header, pages dan footer

Kita akan menggunakan komponen `<Outlet/>` dan menerapkan Nested Routes agar aplikasi kita memiliki 2 layout yang berbeda

▼ Tahap 1 - Main Layout

- Pindahkan komponen `Sidebar.jsx` dan `Header.jsx` ke folder `/components`
- Buat file baru di `/layouts/` dengan nama `MainLayout.jsx` lalu salin tag `<div>` utama yang ada di `App.jsx` ke `MainLayout`
- Hapus komponen `<Routes>` beserta isinya dan tambahkan komponen `<Outlet/>` sebagai gantinya

```

6  export default function MainLayout(){
7    return(
8      <div className="bg-gray-100 min-h-screen flex">
9        <div className="flex flex-row flex-1">
10         <Sidebar />
11         <div className="flex-1 p-4">
12           <Header />
13           <Outlet/>
14         </div>
15       </div>
16     </div>
17   )
18 }
19

```

Jangan lupa import komponen Sidebar, Header dan Outlet



Perhatikan bahwa `Outlet` akan menampilkan komponen halaman yang menjadi turunan dari route yang menggunakan `MainLayout` sebagai elemen pembungkusnya.

- Selanjutnya hapus seluruh tag `<div>` dan komponen di `App.jsx` (karena sudah kita salin ke `MainLayout`) kecuali komponen `<Routes>`, lalu sesuaikan penulisannya menjadi seperti berikut

Before

```

export default function App(){
  return (
    <Routes>
      <Route path="/" element={Dashboard} /> />
      <Route path="orders" element={Orders} /> />
      <Route path="customers" element={Customers} /> />
      <Route path="*" element={NotFound} /> />
    </Routes>
  )
}

```

After

```

export default function App(){
  return (
    <Routes>
      <Route element={MainLayout}>
        <Route path="/" element={Dashboard} /> />
        <Route path="orders" element={Orders} /> />
        <Route path="customers" element={Customers} /> />
        <Route path="*" element={NotFound} /> />
      </Route>
    </Routes>
  )
}

```



Perhatikan Nested Routes diatas, Route seperti `/`, `/orders`, dan `/customers` didefinisikan sebagai child dari route yang memiliki elemen `MainLayout`. Sehingga saat salah satu dari route tersebut diakses, komponen yang dirender akan dibungkus oleh `MainLayout`

Silahkan cek aplikasi dan pastikan tidak ada yang error

▼ Tahap 2 - Auth Layout

Sekarang, kita akan membuat layout khusus untuk menampilkan halaman Autentikasi seperti Login Page, Forgot password, dan Register

- Buat file baru di `/layouts/` dengan nama `AuthLayout.jsx` dan salin kode berikut:

▼ AuthLayout.jsx

```
import { Outlet } from "react-router-dom";

export default function AuthLayout() {
  return (
    <div className="min-h-screen flex items-center justify-center bg-gray-100">
      <div className="bg-white p-8 rounded-2xl shadow-md w-full max-w-md">
        <div className="flex items-center justify-center mb-6">
          <h1 className="text-4xl font-oppins font-extrabold text-gray-800">
            <span className="text-black">Sedap</span>
            <span className="text-green-500">.</span>
          </h1>
        </div>
      </div>
    </div>
  );
}
```

```

        <Outlet/>

        <p className="text-center text-sm
text-gray-500 mt-6">
            © 2025 Sedap Restaurant Admin
            Dashboard. All rights
            reserved.
        </p>
    </div>
</div>
)
}

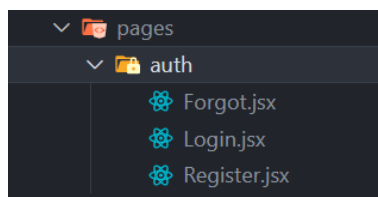
```



Perhatikan bahwa komponen `Outlet` berfungsi untuk merender *child route* yang sesuai ketika `AuthLayout` digunakan. Dengan kata lain, `Outlet` akan menampilkan komponen halaman yang menjadi turunan dari route yang menggunakan `AuthLayout` sebagai elemen pembungkusnya.

- Kita akan menampilkan halaman Login, halaman Forgot Password dan halaman Register.

Buat komponen baru yaitu `Login`, `Register`, `Forgot` dan letak di `/pages/auth`



▼ Login.jsx

```

export default function Login() {
    return (

```

```

        <div>
            <h2 className
            ="text-2xl font-semibold t
            ext-gray-700 mb-6 text-cen
            ter">
                Welcome Ba
                ck 🙌
            </h2>

            <form>
                <div class
                Name="mb-5">
                    <label
                    className="block text-sm f
                    ont-medium text-gray-700 m
                    b-1">
                        Em
                        ail Address
                    </labe
                    l>
                        <input
                        ty
                        pe="text"
                        id
                        ="email"
                        cl
                        assName="w-full px-4 py-2
                        bg-gray-50 border border-g
                        ray-300 rounded-lg shadow-
                        sm
                        placeholder-gray-400"
                        pl
                        aceholder="you@example.co
                        m"
                    />

```

```

        </div>
        <div class
Name="mb-6">
            <label
className="block text-sm font-medium text-gray-700 mb-1">
                Password
            </label>
            <input
                type
                type="password"
                id
                id="password"
                className="w-full px-4 py-2 bg-gray-50 border border-gray-300 rounded-lg shadow-sm"
                placeholder="password"
                placeholder="*****"
            />
        </div>
        <button
            type
            type="submit"
            className="w-full bg-green-500 hover:bg-green-600 text-white font-semibold py-2 px-4 rounded-lg transition duration-200"
        >
            Submit
        </button>
    </div>
</div>

```

```

on-300"
        >
            Login
        </button>
    </form>
</div>
)
}

```

▼ Register.jsx

```

export default function Register() {
    return (
        <div>
            <h2 className="text-2xl font-semibold text-gray-700 mb-6 text-center">
                Create Your Account ✨
            </h2>
            <form>
                <div className="mb-5">
                    <label>
                        Email Address
                    </label>
                    <input type="text" className="block text-sm font-medium text-gray-700 mb-1" />
                </div>
            </form>
        </div>
    )
}

```

```

    Address
    </label>
  </div>
  <input
    type="email"
    id="email"
    class="w-full px-4 py-2 bg-gray-50 border border-gray-300 rounded-lg shadow-sm placeholder-gray-400"
    placeholder="you@example.com"
  />
</div>

<div class="mb-5">
  <label
    for="password"
    class="block text-sm font-medium text-gray-700 mb-1"
  >
    Password
  </label>
  <input

```



```

        type="password"
        id="password"
        className="w-full px-4 py-2 bg-gray-50 border border-gray-300 rounded-lg shadow-sm"
        placeholder="password"
        placeholder="*****"
    />
</div>

<div class="mb-6">
    <label
        htmlFor="confirmPassword"
        className="block text-sm font-medium text-gray-700 mb-1"
    >
        Confirm Password
    </label>
    <input
        type="password"
        id="confirmPassword"
        class=

```

```

    className="w-full px-4 py-2
    bg-gray-50 border border-g
    ray-300 rounded-lg shadow-
    sm

    placeholder-gray-400"
    type="password"
    value=""
  </div>

  <button
    type="submit"
    className="w-full bg-green-500 h
    over:bg-green-600 text-whi
    te font-semibold py-2 px-4
    rounded-lg transition durati
    on-300"
  >
    Register
  </button>
</form>
</div>
)
}

```

▼ Forgot.jsx

```

export default function Fo
rgot() {
  return (

```

```

        <div>
            <h2 className
            ="text-2xl font-semibold t
            ext-gray-700 mb-2 text-cen
            ter">
                Forgot Your
                Password?
            </h2>

            <p className
            ="text-sm text-gray-500 mb
            -6 text-center">
                Enter your
                email address and we'll se
                nd you a link to reset you
                r
                password.
            </p>

            <form>
                <div class
                Name="mb-5">
                    <label
                    ht
                    mlFor="email"
                    cl
                    assName="block text-sm fon
                    t-medium text-gray-700 mb-
                    1"
                    >
                        Em
                        ail Address
                    </labe
                    l>

                    <input
                    ty

```

```

    type="email"
    id
    ="email"
    cl
    className="w-full px-4 py-2
    bg-gray-50 border border-g
    ray-300 rounded-lg shadow-
    sm

    placeholder-gray-400"
    pl
    placeholder="you@example.co
    m"
    />
  </div>
  <button
    type
    ="submit"
    classN
    ame="w-full bg-green-500 h
    over:bg-green-600 text-whi
    te font-semibold py-2 px-4
    ro
    unded-lg transition durati
    on-300"
    >
    Send R
    eset Link
  </button>
</form>
</div>
)
}

```

- Tambahkan di `App.jsx` komponen `Route` baru dengan property elemen yang menerapkan `AuthLayouts` sebagai pembungkusnya.

maka buat Route baru dengan path `/login` , `/forgot` dan `/register` beserta element nya

```
<Routes>
  /*Route MainLayout*/
  ...

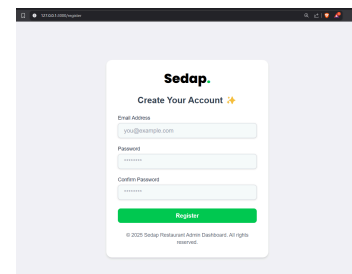
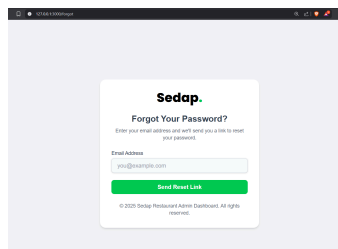
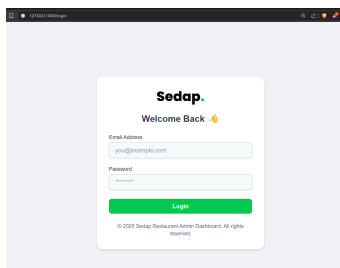
  <Route element={<AuthLayout/>}>
    <Route path="/login" element={<Login />}
  />

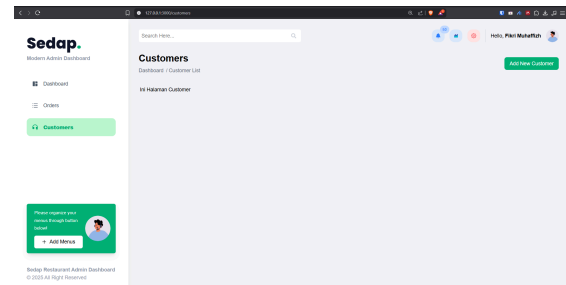
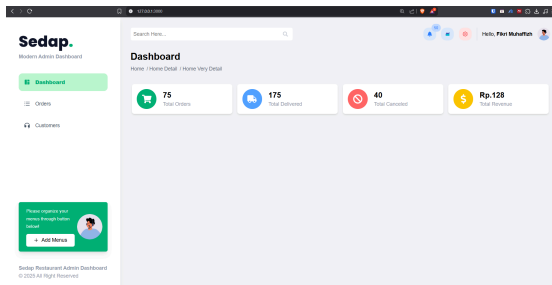
    <Route path="/register" element={<Register/>} />

    <Route path="/forgot" element={<Forgot/>}
  />

  </Route>
</Routes>
```

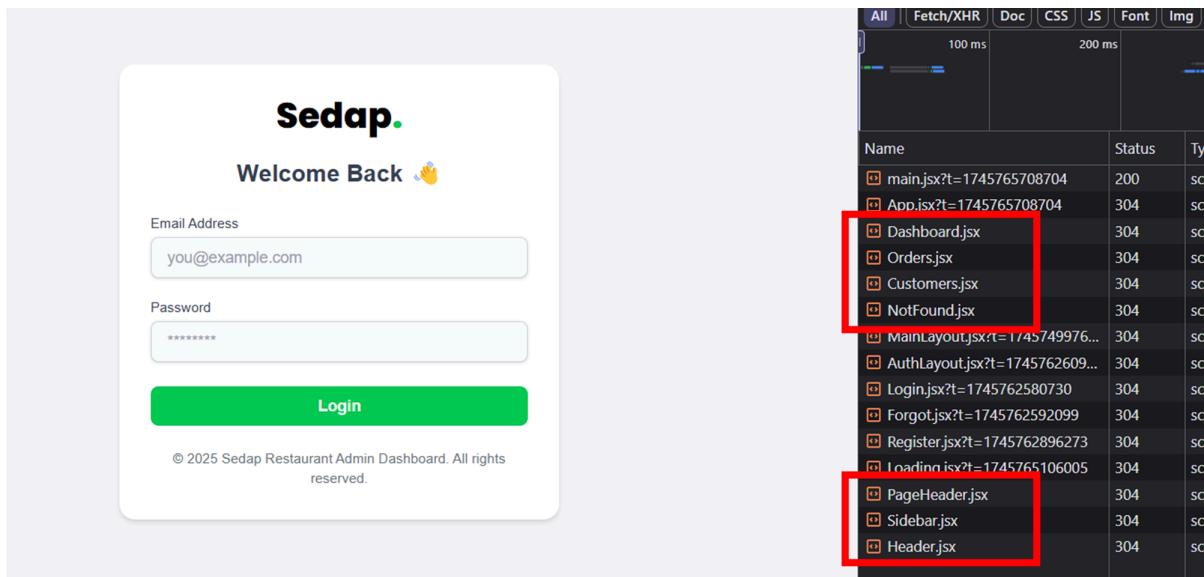
Silahkan akses ke halaman Auth (`/login` , `/register` , `/forgot`) dan halaman Dashboard (`/` , `/orders` , `/customers`), terlihat aplikasi sudah bisa meng-handle 2 layout yang berbeda





▼ Lazy Loading + Suspense

Saat ini, jika kita mengakses halaman `/login` dan kita lihat pada inspect → tab network, terlihat bahwa meski hanya mengakses halaman login, komponen yang di-load tidak hanya Login, namun juga komponen yang tidak kita butuhkan saat ini seperti Dashboard, Customer, Sidebar dll



Hal ini tentu tidak akan masalah jika aplikasi hanya memiliki beberapa halaman. Namun bagaimana jika kita sudah memiliki 20 atau lebih komponen halaman ? tentu akan mempengaruhi performa aplikasi

Kita akan menerapkan `React.Lazy()` yang akan meng-import komponen secara dinamis hanya saat dibutuhkan. Silahkan ubah pemanggilan `import` komponen yang ada seperti berikut:

Before

```
import Dashboard from "../pages/Dashboard"
```

After

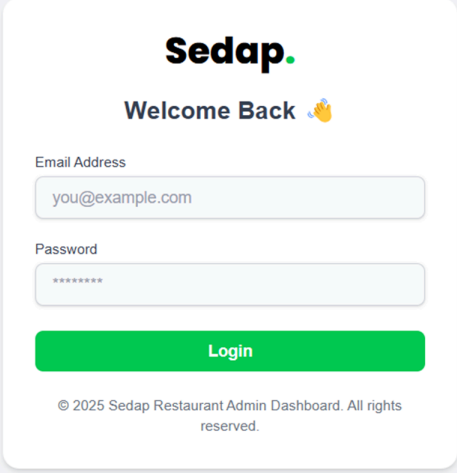
```
const Dashboard = React.lazy(()  
=> import("../pages/Dashboard"))
```

Pastikan di-import komponen `React`



`React.lazy()` sebaiknya digunakan pada komponen yang jarang digunakan atau besar. Tidak perlu diterapkan di seluruh komponen, cukup pada komponen parent seperti yang ada di folder `/pages`.

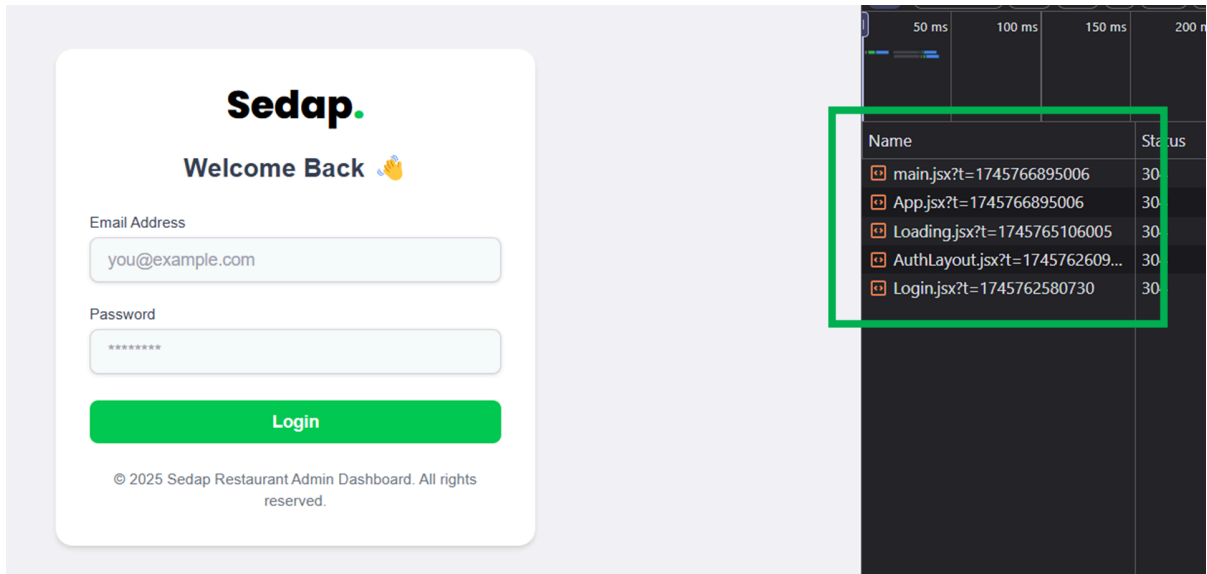
Silahkan cek kembali saat mengakses `/login`, komponen `Dashboard` seharusnya sudah tidak di-load lagi



Komponen **Dashboard** sudah tidak di-load

Name	Status	Type
main.jsx?t=1745766477147	200	scr
App.jsx?t=1745766477147	200	scr
Orders.jsx	304	scr
Customers.jsx	304	scr
NotFound.jsx	304	scr
MainLayout.jsx?t=1745749976...	304	scr
AuthLayout.jsx?t=1745752609...	304	scr
Login.jsx?t=1745762581730	304	scr
Forgot.jsx?t=17457625812099	304	scr
Register.jsx?t=174576258196273	304	scr
Loading.jsx?t=174576506005	304	scr
PageHeader.jsx	304	scr
Sidebar.jsx	304	scr
Header.jsx	304	scr

Jika sudah diterapkan ke komponen lain di `App.jsx`, kita bisa optimasi halaman login seperti berikut



Saat menerapkan `React.Lazy()` , kita bisa menerapkan Loading saat menampilkan komponen. Kita akan coba menerapkan Loading pada komponen utama , silahkan bungkus komponen `<Routes>` dengan `<Suspense>` beserta proposer fallback yang memanggil komponen `Loading`

```
export default function App() {
  return (
    <Suspense fallback={<Loading />}>
      <Routes>
        ...
      </Routes>
    </Suspense>
  )
}
```

Silahkan buat file `Loading.jsx` di `/components` dan salin kode berikut

▼ Loading.jsx

```
export default function Loading() {
  return (
    <div className="flex flex-col justify-center items-center min-h-screen bg-white">
      <div className="w-12 h-12 border">
        ...
      </div>
    </div>
  )
}
```



```

    )
  }

```

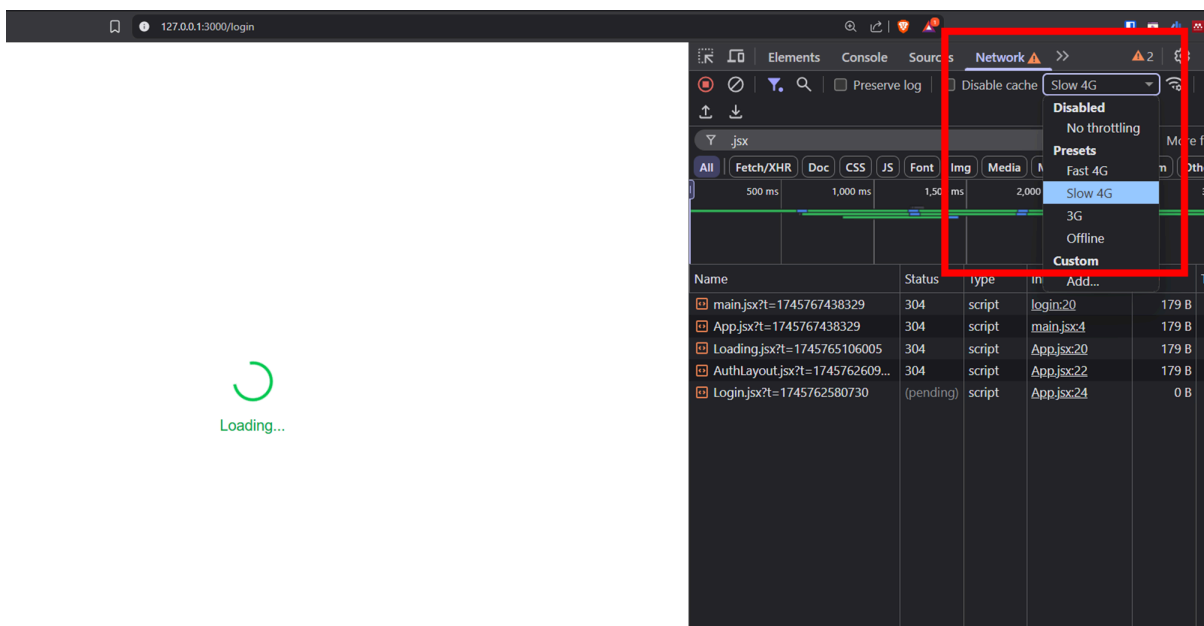
```

-4 border-green-500 border-t-transparent rounded-full animate-spin mb-4"></div>
      <p className="text-green-600 text-lg">Loading...</p>
    </div>
  );
}

```

Silahkan tes dan pastikan mengganti jaringan di inspect → Network menjadi "Slow 4G" untuk mensimulasikan koneksi lambat dan tampilnya komponen

Loading



▼ Login API + Axios

Kita akan coba menerapkan logic Login pada kompone `Login` dengan memanfaatkan public endpoint yaitu:

<https://dummyjson.com/docs/users#users-login>

Jika kita perhatikan pada dokumentasi diatas, kita bisa menggunakan endpoint

`/users/login` dengan mengirimkan `username` dan `password`

Login user and get tokens [↗](#)

💡 You can use any user's credentials from dummyjson.com/users. Tok

```
fetch('https://dummyjson.com/user/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    username: 'emilys',
    password: 'emilypass',
    expiresInMins: 30, // optional, defaults to 60
  })
})
.then(res => res.json())
.then(console.log);
```

`username` : emilys

`password` : emilypass

```
{
  "id": 1,
  "username": "emilys",
  "email": "emily.johnson@x.dummyjson.com",
  "firstName": "Emily",
  "lastName": "Johnson",
  "gender": "female",
  "image": "https://dummyjson.com/icon/emilys/128",
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Response yang akan diterima

Kita akan modifikasi komponen `Login` yang sudah kita buat sebelumnya.
Silahkan ikut langkah berikut:

1. Install package `Axios` terlebih dahulu

```
npm install axios
```

2. Kita definisikan beberapa hal berikut:

- `navigate` → untuk **berpindah halaman** ke halaman `Dashboard` setelah login berhasil.
- `loading & error` → untuk **menyimpan status** dan mengontrol tampilan komponen **loading** (permintaan sedang diproses) dan **error** (pesan kesalahan jika terjadi masalah).
- `dataForm & handleChange` → untuk **menyimpan nilai inputan username** dan **password** serta **mengupdate** state ketika ada perubahan pada inputan tersebut.

▼ **Klik Disini**

```

export default function Login() {
  /* navigate, state & handleChange*/
  const navigate = useNavigate()
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState("")
  const [dataForm, setDataForm] = useState({
    email: "",
    password: "",
  })

  const handleChange = (evt) => {
    const { name, value } = evt.target
    setDataForm({
      ...dataForm,
      [name]: value,
    })
  }

  ...

  return(
    ...
  )
}

```

3. Definisikan `handleSubmit` untuk meng-handle proses send data ke endpoint

▼ **Klik Disini**

```

export default function Login() {
  /* navigate, state & handleChange*/
  ...

  /* process form */

```

```

const handleSubmit = async (e) => {
    e.preventDefault()

    setLoading(true)
    setError(false)

    axios
        .post("https://dummyjson.com/user/login", {
            username: dataForm.email,
            password: dataForm.password,
        })
        .then((response) => {
            // Jika status bukan 200, tampilkan pesan error
            if (response.status !== 200) {
                setError(response.data.message);
                return;
            }

            // Redirect ke dashboard jika login sukses
            navigate("/");
        })
        .catch((err) => {
            if (err.response) {
                setError(err.response.data.message || "An error occurred");
            } else {
                setError(err.message || "An unknown error occurred");
            }
        })
        .finally(() => {

```

```

        setLoading(false);
    });
}

```

4. Karena kita menggunakan state untuk `loading` dan `error`, kita akan membuat variabel untuk mengontrol tampilan kedua komponen tersebut berdasarkan nilai state-nya

▼ **Klik Disini**

```

export default function Login() {
    /* navigate, state & handleChange */
    ...

    /* process form */
    ...

    /* error & loading status */
    const errorInfo = error ? (
        <div className="bg-red-200 mb-5 p-5 text-sm font-light text-gray-600 rounded flex items-center">
            <BsFillExclamationDiamondFill className="text-red-600 me-2 text-lg" />
            {error}
        </div>
    ) : null

    const loadingInfo = loading ? (
        <div className="bg-gray-200 mb-5 p-5 text-sm rounded flex items-center">
            <ImSpinner2 className="me-2 animate-spin" />
            Mohon Tunggu...

```

```
    </div>  
  ) : null
```

5. Kita sesuaikan form `Login` dengan menerapkan `errorInfo`, `loadingInfo`, `handleSubmit` dan `handleChange`

▼ **Klik Disini**

```
return (  
  <div>  
    <h2 className="text-2xl font-semibold text-gray-700 mb-6 text-center">  
      Welcome Back 🙌  
    </h2>  
  
    {errorInfo}  
  
    {loadingInfo}  
  
    <form onSubmit={handleSubmit}>  
      ...  
      <input  
        ...  
        name="email"  
        onChange={handleChange}  
      />  
      ...  
      <input  
        ...  
        name="password"  
        onChange={handleChange}  
      />  
    </form>  
  )
```

```
)  
</div>
```

Berikut hasil implementasi penggunaan Axios untuk melakukan pemanggilan API, termasuk pengelolaan status error dan loading

The image displays three sequential screenshots of a login form for 'Sedap'. Each screenshot shows the 'Sedap.' logo, a 'Welcome Back' message with a thumbs-up icon, and a 'Login' button. The form includes fields for 'Email Address' and 'Password'.
1. The first screenshot shows a loading state with a grey bar at the top containing a loading spinner and the text 'Mohon Tunggu...'. The email field contains '123' and the password field contains three dots.
2. The second screenshot shows an error state with a red bar at the top containing a red diamond icon and the text 'Username and password required'. The email field contains 'you@example.com' and the password field contains eight dots.
3. The third screenshot shows another error state with a red bar at the top containing a red diamond icon and the text 'Invalid credentials'. The email field contains '123' and the password field contains three dots.
All screenshots include a footer: '© 2025 Sedap Restaurant Admin Dashboard. All rights reserved.'

Jika login berhasil, maka akan diarahkan ke halaman Dashboard